Tokens, Context Windows & Costs

How AI reads, remembers, and bills you

Introduction to Generative AI



What Is a Token?



When you message Claude or ChatGPT, the model does not read word by word



It breaks your text into small chunks called tokens



Rule of thumb: 1 token ≈ 3–4 characters of English text



Short, common words = usually 1 token



Longer or rarer words = split into several tokens

Tokenisation Example: A Simple Sentence

Sentence: "I love pizza!"

Chunk	Token #
I	1
love	2
pizza	3
!	4

13 characters → 4 tokens

Tokenisation Example: A Complex Word

Word: "Unbelievable"

Chunk	Token #
Un	1
bel	2
iev	3
able	4

1 word → 4 tokens, because it is long and less common

Which Words Cost More Tokens?

Common words like "the", "is", "a"

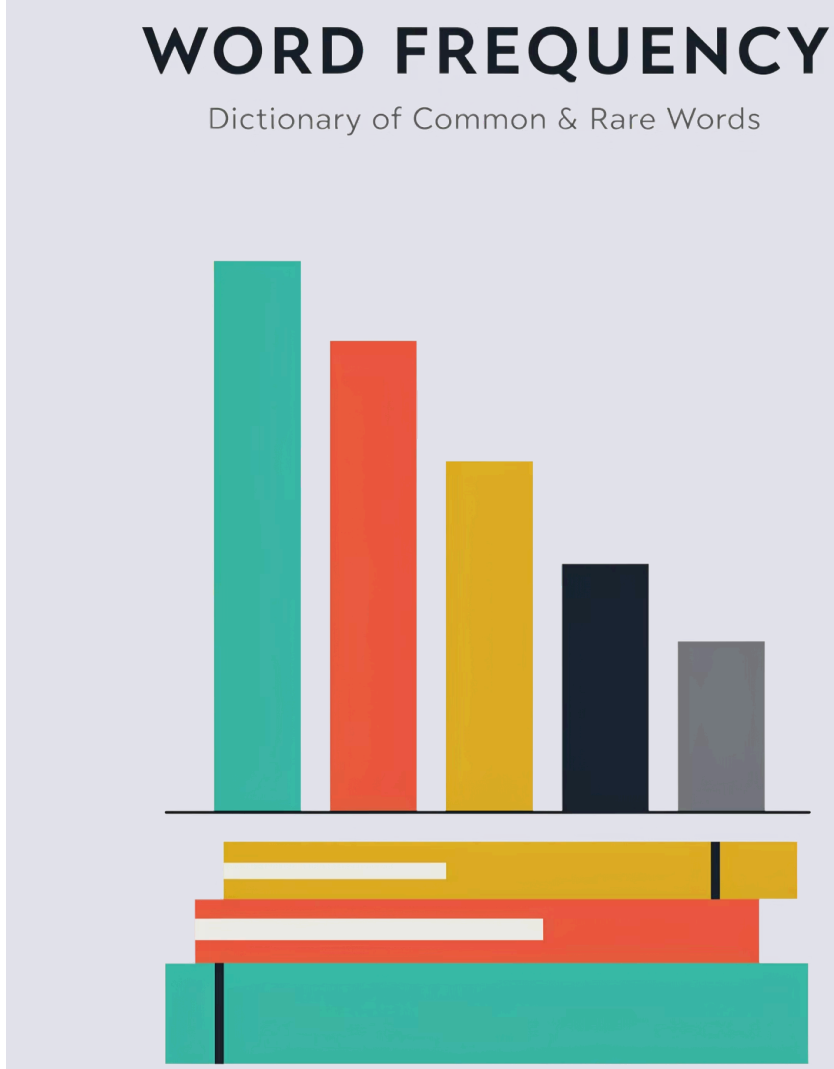
Usually **1 token**

Rare, technical, or very long words

More tokens

Takeaway

Token count is about chunks, not words

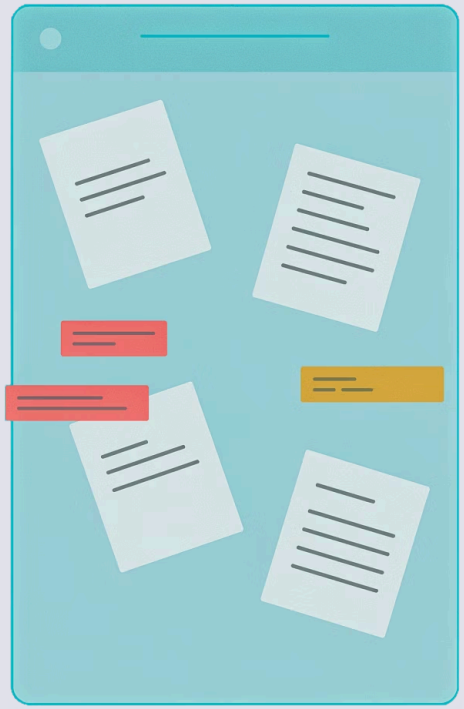


What Is a Context Window?

The AI's short-term memory

The maximum amount of text it can hold in mind at once

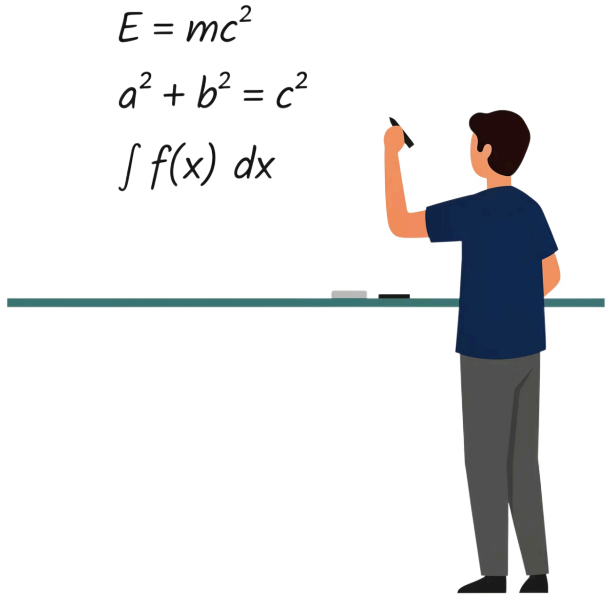
Measured in tokens, not words or pages



Analogy: The Whiteboard

A student solving a maths problem on a whiteboard:

- The board has limited space
- The student can only refer to what is written **on the board**
- When the board fills up, old content must be erased



The context window works exactly the same way.

Inside the Context Window

Example: a 200,000-token window holds everything together:



System instructions



Your conversation



Uploaded documents



The AI's response

Everything here = tokens being used

If conversation + documents exceed the limit, the model either **forgets** earlier parts or refuses to proceed.



Case Study: Summarising a Textbook

A student uploads a 100-page chemistry PDF and asks:

> "Summarise Chapter 5 and answer my 3 questions."

What happens under the hood?

Case Study: Token Budget

Content	Estimated Tokens
Entire 100-page textbook	~75,000
Student's questions	~80
System instructions	~500
Total	**~75,580**

- 128,000-token window → fits, AI reads everything
- 32,000-token window → overflows, earlier chapters are **not visible**

Warning: Silent Failure

When a document exceeds the context window, the AI doesn't warn you loudly. It simply works with what fits.

Result: incomplete or inaccurate answers, with no obvious error message.



Two Kinds of Tokens

Every AI interaction has tokens flowing in **opposite directions**:

Input tokens (prompt tokens)

You → AI

Response tokens (output / completion tokens)

AI → You

Input Tokens

Everything the model reads **before** it replies:

- **Your question or instruction**
- **Documents, code, or data you paste or upload**
- **The system prompt (hidden instructions set by the app)**
- **The entire conversation history so far**

Response Tokens

Everything the AI **generates**, one token at a time:

- **The answer, explanation, or summary**
- **Any code it writes**
- **Tables, lists, and structured output**

Input vs Response Tokens

	Input Tokens	Response Tokens
Direction	You → AI	AI → You
Includes	Prompt, documents, history, system instructions	The AI's full reply
Who controls it?	You	Partly you (max_tokens), partly the model
Predictable?	Yes, you can count before sending	No, depends on the question
Cost	Cheaper per token	More expensive per token
Speed impact	Longer input = longer processing	Longer output = longer streaming time

Analogy: Exam Paper vs Answer Sheet

Input tokens

= the exam question paper (everything the student reads)

Response tokens

= the student's written answer (everything produced)

The paper can be long; the answer can be short or long.

Both use paper and ink, and you are billed separately for each.

Worked Example: Input Side

A student asks for help with physics:

Part	Tokens
System prompt: "You are a helpful tutor."	8
History: previous 3 exchanges	320
Question: "Explain Newton's 2nd law with a real-life example."	18
Total input	**346**

Worked Example: Output Side

The AI replies with a 3-paragraph explanation (~210 words):

280	346	626
Response tokens	**Input tokens**	**Total billed tokens**

Total billed = 346 (input) + 280 (output) = 626 tokens

Note: after 10 back-and-forth exchanges, history alone can reach **3,000–5,000 tokens**.

How Token Pricing Works

Providers (Anthropic, OpenAI, Google) charge **separately** for input and output

Output is usually priced **higher**

Why? Each output token needs a full pass through billions of parameters, one token at a time. Reading input is comparatively cheaper.



Illustrative Pricing

Approximate figures for illustration. Actual prices vary by model and provider.

Model Tier	Input (per 1M tokens)	Output (per 1M tokens)
Small / Fast	$0.25	$1.00
Mid-tier	$3.00	$15.00
Large / Flagship	$15.00	$75.00

Real Cost Scenario: Grading 30 Essays

Teacher uses a mid-tier model, essays ~500 words each:

Component	Tokens	Cost
30 essays as input (30 × ~700)	21,000	$0.063
Feedback (~300 tokens × 30)	9,000	$0.135
System prompt + overhead	1,000	$0.003
Total	**31,000**	**~$0.20 (≈ ₹17)**

Detailed feedback on 30 essays for about the price of a sweet. Extremely economical.

What Is "System Prompt + Overhead"?

System prompt

Hidden instructions sent *before* your message, written by the app or developer. It sets tone, role, restrictions, and formatting.

Example: "You are a helpful tutor. Always explain in simple English. Never give direct answers to homework."

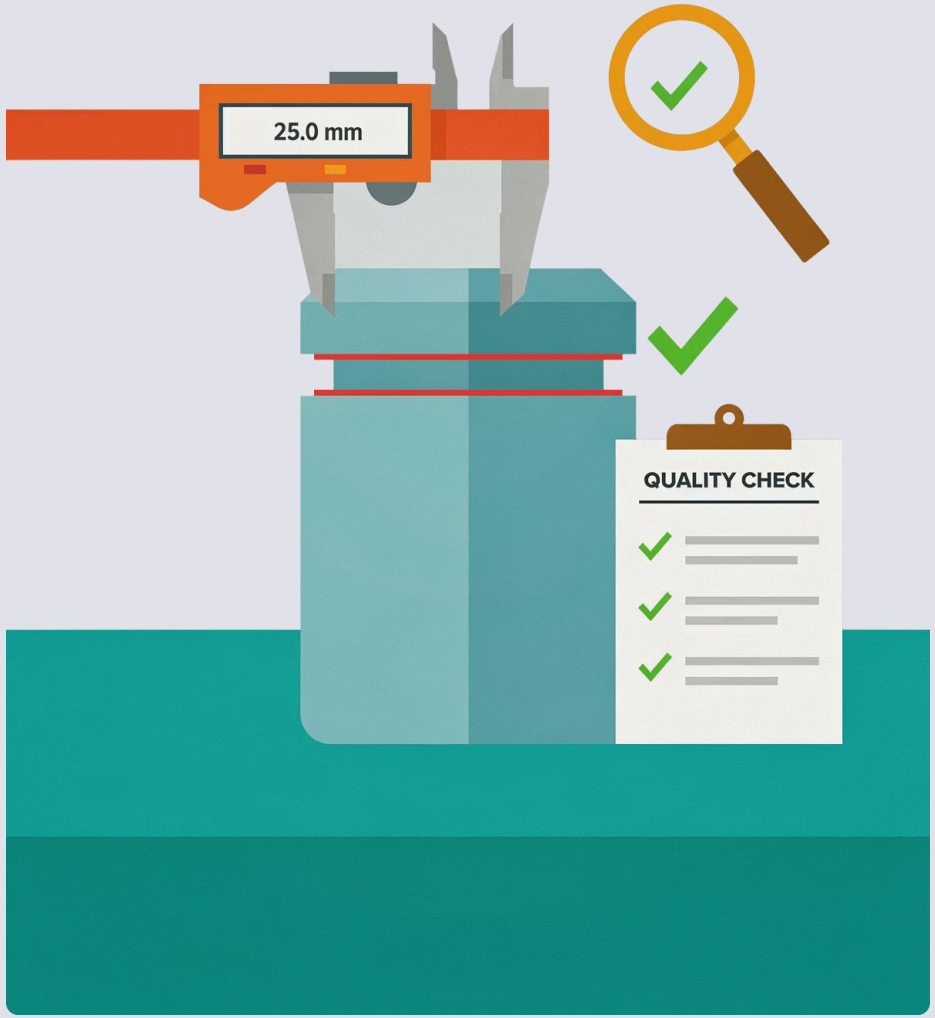
Overhead

Extra tokens the platform adds automatically, such as safety filters, metadata, tool descriptions, and API scaffolding.

These background tokens count toward your context window **and** your bill on every request.

Tips to Manage Token Costs

Strategy	Why It Helps
Trim your prompt	Fewer input tokens = direct savings
Ask for concise replies	"Answer in 3 bullet points" cuts output tokens
Don't repeat context	Avoid pasting the same document multiple times
Use a smaller model when appropriate	A fast model at 1/10th the price is often good enough
Set max_tokens	Caps output length and prevents costly surprises



Why Does This Matter for Quality?

Context window limits affect output quality in **two ways**:

1. Recency bias ("Lost in the Middle")

2. Context pollution

Lost in the Middle



- Research shows models pay **more attention** to the very **start** and very **end** of the context
- Information buried in the middle gets **underweighted**

Practical tip: put your most important instructions at the **top and bottom** of your prompt.

Context Pollution

The Problem

Filling the window with irrelevant content (e.g. a whole book when you need one chapter) leaves less room for careful reasoning

Quality degrades when the model is overwhelmed with noise

The Solution

Send what's relevant, not everything you have.



Key Takeaways

Concept	Remember
Token	A small chunk of text (~3–4 characters); the AI's basic unit of reading and writing
Input tokens	Everything you send: prompt, documents, history, system instructions
Response tokens	Everything the AI writes back; generated one at a time; usually priced higher
Context window	The AI's working memory; input + response must both fit
Cost	Billed separately; output costs more; concise prompts save money
Quality	Overfilling or burying key info in the middle reduces accuracy



Rule of Thumb

1 page of text ≈ 500–750 tokens

Use this to estimate whether your content fits a model's context window.

Quick Check: Discuss in Pairs

1

Why might "Unbelievable" cost more tokens than "the"?

2

A 100-page PDF is ~75,000 tokens. Will it fit in a 32,000-token window? What happens?

3

Why does a long chat get more expensive with every message?

4

Where should you place your most important instruction in a long prompt?